

Practica Patrón State

Víctor Manuel Peiró Martínez

FIIS 2ºA

Diseño De Software

Solución:

Introducción:

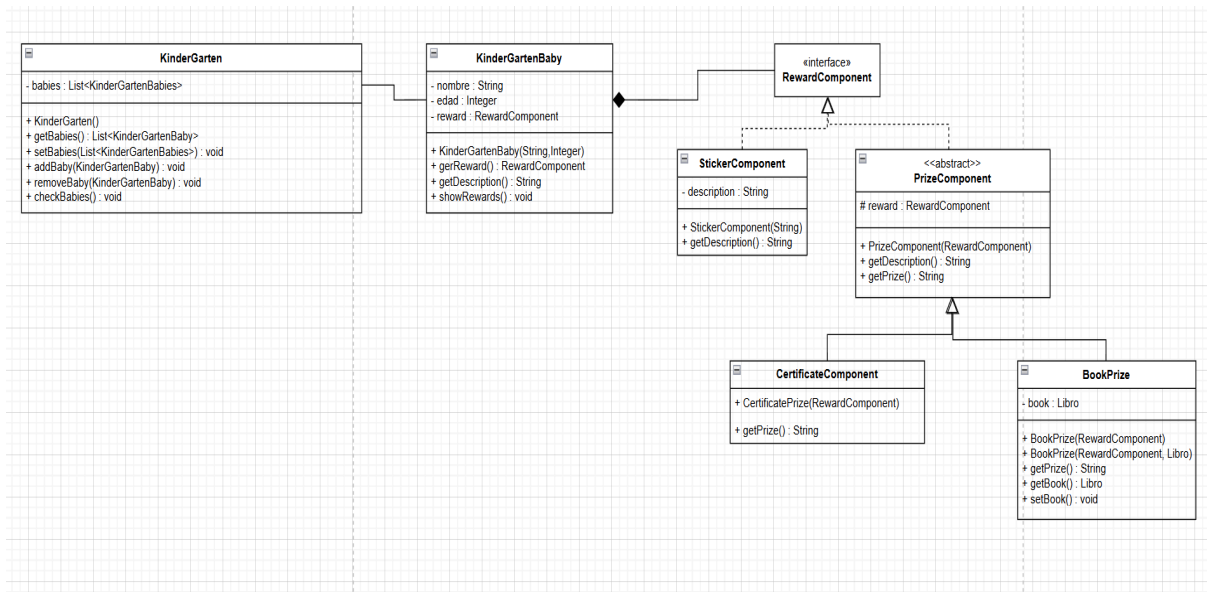
Estamos modelando una guardería donde los niños reciben premios en función de su comportamiento en clase. Hay 3 tipos de premios:

- Pegatinas: Se les da a todos los alumnos sin importar su comportamiento
- Certificados: Se le da a todos los niños que van al baño en vez de hacer sus necesidades en la pañal
- Libros: Se les da a los alumnos escogidos como alumno de la semana

Para ello, aplicaremos el patrón de diseño decorator donde decoramos los premios que se lleven los niños. El patrón decorator está definido por varios componentes cual explicaremos ahora y veremos donde están implementados:

- **Componente:** Esta es la interfaz que define a los objetos que pueden ser decorados. En nuestra implementación es la interfaz **RewardComponent**
- **Componente Base:** Es la clase que vamos a decorar. En nuestra implementación es la clase **StickerComponent**. He decidido que la pegatina se el componente base ya que es el premio que recibe todo el mundo y así se podrá decorar añadiendo mas premios
- **Decorator:** Clase Abstracta con referencia al componente que define los metodos que tendran los decoradores concretos. En nuestra implementación es la clase **PrizeComponent** que tiene el metodo para concatenar a los distintos premios y un método abstracto para ser definido en cada decorador concreto
- **Decorator Concreto:** Añade funcionalidad a cada decorador que haga cosas distintas. En nuestro caso son las clases:
 - **BookPrize**
 - **CertificatePrize**

UML:



En .zip se adjunta archivo draw.io

Código:

Componente y Componente Base

```
package EntregaDecoratorPattern;
//Interfaz componente
public interface RewardComponent {
    public String getDescription();
}
```

```
package EntregaDecoratorPattern;
//Componente Base
public class StickerComponent implements RewardComponent{

    //Atributo --> Descripcion del premio
    private String description;

    //Constructor
    public StickerComponent(String description) {
        this.description = description;
    }

    //Devuelve la descripcion del premio
    @Override
    public String getDescription() {
        return this.description;
    }
}
```

Decoradores

```
package EntregaDecoratorPattern;
//Decorador Abstracto
public abstract class PrizeComponent implements RewardComponent{

    //Atributo --> Componente a decorar
    protected RewardComponent reward;

    //Constructor
    public PrizeComponent(RewardComponent reward) {
        this.reward = reward;
    }

    public abstract String getPrize(); //Metodo abstracto a definir en decoradores concretos
}
```

```

public String getDescription() {
    String descripcion = "";

    //Si el niño ya tiene premios, hemos de concatenar las nuevas cadenas a para
    que se vea que tiene mas de un premio
    if (this.reward != null) {
        descripcion = this.reward.getDescription();
    }

    //Añado el nuevo premio
    descripcion += this.getPrize();

    return descripcion;
}
}

```

```

package EntregaDecoratorPattern;
import EntregaObserverPattern.Libro;
//Decorador concreto del libro
public class BookPrize extends PrizeComponent{

    //Atributo
    private Libro book;

    //Constructor con libro por defecto. En este caso es el Quijote
    public BookPrize(RewardComponent reward) {
        this(reward,new Libro());
    }

    //Constructor
    public BookPrize(RewardComponent reward,Libro book) {
        super(reward);
        this.book = book;
    }

    //Devuelve la descripcion del premio por ser niño/a de la semana
    @Override
    public String getPrize() {
        return " es niño/a de la semana con libro: " + this.book.toString();
    }

    //Getters y Setters

```

```

    public Libro getBook() {
        return book;
    }

    public void setBook(Libro book) {
        this.book = book;
    }
}

```

```

package EntregaDecoratorPattern;
//Decorador Concreto para certificado
public class CertificatePrize extends PrizeComponent{

    //Constructor
    public CertificatePrize(RewardComponent reward) {
        super(reward);
    }

    //Devuelve la descripcion de premio
    @Override
    public String getPrize() {
        return " con premio de buena conducta pañal";
    }
}

```

Clase KindergartenBaby

```

package EntregaDecoratorPattern;
public class KindergartenBaby {

    //Atributos
    private String nombre;
    private Integer edad;
    private RewardComponent reward;

    //Constructor
    public KindergartenBaby(String nombre, Integer edad) {
        this.nombre = nombre;
        this.edad = edad;
        this.reward = new StickerComponent(this.nombre);
    }

    //Getters y Setters

```

```

    public String getNombre() {
        return this.nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public Integer getEdad() {
        return this.edad;
    }
    public void setEdad(Integer edad) {
        this.edad = edad;
    }
    public RewardComponent getReward() {
        return this.reward;
    }
    public void setReward(RewardComponent reward) {
        this.reward = reward;
    }
}

    public String getDescription() {
        return this.reward.getDescription(); //Delegacion por composicion
    }

    //Muestra las recompensas de cada bebe
    public void showRewards() {
        System.out.println(this.getDescription());
    }
}

```

Clase KinderGarten

```
package EntregaDecoratorPattern;
```

```
import java.util.List;
```

```
import java.util.ArrayList;
```

```
//Clase Guarderia
```

```
public class Kindergarten {
```

```
    //Atributo --> Lista de bebes
```

```
    private List<KinderGartenBaby> babies;
```

```

//Constructor
public Kindergarten() {
    this.babies = new ArrayList<KinderGartenBaby>();
}

//Getters y Setters
public List<KinderGartenBaby> getBabies() {
    return this.babies;
}

public void setBabies(List<KinderGartenBaby> babies) {
    this.babies = babies;
}

//Añade un bebe a la lista
public void addBaby(KinderGartenBaby baby) {
    this.babies.add(baby);
}

//Elimina un bebe de la lista
public void removeBaby(KinderGartenBaby baby) {
    this.babies.remove(baby);
}

//Muestra los premios de todos los bebes
public void checkBabies() {
    for (KinderGartenBaby baby : this.babies) {
        baby.showRewards();
    }
}
}

```

Clase Test

```

package EntregaDecoratorPattern;
public class KindergartenTest {
    public static void main(String[] args) {
        Kindergarten kinderGarten = new Kindergarten();

        //Creamis los bebes
        KinderGartenBaby ines = new KinderGartenBaby("Ines", 1);
        KinderGartenBaby kike = new KinderGartenBaby("Kike", 2);

        //Añadimos los bebes a la guarderia
        kinderGarten.addBaby(ines);
    }
}

```



```

kinderGarten.addBaby(kike);

//Pasamos lista
kinderGarten.checkBabies();

//Le damos una certificado a Ines
ines.setReward(new CertificatePrize(ines.getReward()));

//Pasamos lista
kinderGarten.checkBabies();

//Ines es la niña de la semana por ello recibe un libro
BookPrize premioLibro = new BookPrize(ines.getReward()); //Libro por
defecto = El quijote
ines.setReward(premioLibro);

//Pasamos lista
kinderGarten.checkBabies();

//Imprime el nombre del libro de Ines
System.out.println(ines.getNombre() + " tiene el libro: " +
premioLibro.getBook().getNombre());
    }
}

```

Resultado:

```

Ines
Kike
Ines con premio de buena conducta pañal
Kike
Ines con premio de buena conducta pañal es niño/a de la semana con libro: Libro [nombre=El Quijote De La Mancha, autor=Miguel de Cervantes, numPaginas=844, estado=UNKNOWN]
Kike
Ines tiene el libro: El Quijote De La Mancha

```